Grokking Algorithms – Day 1 Notes

Chapters 1 & 2 (up to Selection Sort)

Independent version – written without reference to any practice file

---

1. Binary Search

Binary search is an efficient algorithm for finding an item in a **sorted** list.

How it works

- Look at the middle element.
- If it is the target → done.
- If the target is smaller → discard the right half.
- If the target is larger → discard the left half.
- Repeat until the item is found or the search space is empty.

```
def binary_search(list, item):
    low = 0
    high = len(list) - 1

    while low <= high:
        mid = (low + high) // 2
        guess = list[mid]

        if guess == item:
            return mid
        if guess > item:
            high = mid - 1
        else:
            low = mid + 1

    return None   # item not found
```

Time complexity: O(log n)

Quick exercises

- List of 128 sorted names → maximum steps = **7**
- List of 256 sorted names → maximum steps = **8**

Binary search is much faster than simple (linear) search once the list becomes large.

2. Big O Notation

Big O tells us how the runtime of an algorithm grows as the size of the input grows. We care about the **growth rate**, not the exact number of seconds.

Common runtimes (fastest → slowest)

Big O	Name	Typical use case
O(1)	Constant	Accessing an array element
O(log n)	Logarithmic	Binary search
O(n)	Linear	Simple search / looping once
O(n log n)	Log-linear	Efficient sorting algorithms
O(n^2)	Quadratic	Selection sort, nested loops
O(n!)	Factorial	Brute-force traveling salesperson

Phone book examples

- Looking up a name to get a phone number → O(log n) (can use binary search)
- Looking up a phone number to get a name → O(n) (must scan the whole book)
- Reading every number in the book → O(n)
- Reading only the numbers that start with “A” → still O(n) in the worst case

3. Arrays vs Linked Lists

Memory is like a huge row of drawers. How you organize data in those drawers matters a lot.

Array

An **array** stores elements in contiguous (side-by-side) memory locations. Because the elements sit next to each other, you can jump directly to any index in constant time.

In Python we usually use a list to represent an array:

```
# Simple array (list) of numbers
arr = [10, 20, 30, 40, 50]

# Access by index is instant
print(arr[2])   # → 30
```

Linked List

A **linked list** stores elements as separate nodes. Each node holds a value and a pointer (reference) to the next node. The nodes can be scattered anywhere in memory; they are only connected by the pointers.

In Python a basic linked list looks like this:

```
class Node:
    def __init__(self, value):
        self.value = value
        self.next = None

# Build a short linked list: 10 → 20 → 30
head = Node(10)
head.next = Node(20)
head.next.next = Node(30)
```

To reach the third element you must start at the head and follow the `next` pointers one by one.

Comparison

Operation	Array	Linked List
Get element by index	O(1)	O(n)
Insert / delete at beginning	O(n)	O(1)
Insert / delete at end	O(1) amortized	O(1) (with tail pointer)
Insert in the middle	O(n)	O(1) (if you have the node)

When to use which

- **Array** → You need fast random access (example: binary search on a list of usernames).
- **Linked List** → You do many insertions and deletions (example: a queue of restaurant orders).

Important trade-off: If you store usernames in an array so you can binary-search them quickly, every new signup becomes expensive: you must keep the array sorted, which means shifting many elements (O(n)).

4. Selection Sort

Selection sort repeatedly finds the smallest remaining item and moves it into its final position.

Algorithm

1. Find the smallest element in the list.
2. Swap it with the element at the front of the unsorted portion.
3. Move the boundary of the unsorted portion one step forward.
4. Repeat until the whole list is sorted.

```
def find_smallest(arr):
    smallest = arr[0]
    smallest_index = 0
    for i in range(1, len(arr)):
        if arr[i] < smallest:
            smallest = arr[i]
            smallest_index = i
    return smallest_index

def selection_sort(arr):
    new_arr = []
    for i in range(len(arr)):
        smallest = find_smallest(arr)
        new_arr.append(arr.pop(smallest))
    return new_arr
```

Complexity

- Time: $O(n^2)$
- Space: O(n) (this version creates a new list)

Selection sort is easy to understand but slow for large lists. It is mainly useful as a teaching example.

Day 1 Recap

- Binary search needs a sorted list and gives O(log n) performance.
- Big O describes growth rate, not absolute time.
- Arrays give fast reads; linked lists give fast inserts/deletes.
- Selection sort is simple and $O(n^2)$.

Notes based solely on Grokking Algorithms, Chapters 1–2 (up to Selection Sort).